Technical report: Booklet NLI, claim checking over Swiss voting booklets with Apertus

- **Track:** Track 2A, OST challenge: *Multilingual Natural Language Inference over Swiss Official Voting Booklets*
- **Event:** Online
- **Team:** kaiser-data (Martin Kaiser)
- **Code:** https://github.com/kaiser-data/hack-apertus-booklet-nli (`track_2a/`)
- **Visual summary:** `track_2a/docs/results.html`

1. Summary

Given an official Swiss federal voting booklet (German, French or Italian PDF) and a claim in any of the three languages, the system decides **entailment, neutral or contradiction** and cites the booklet passages, with page numbers, that the decision rests on. A small multilingual embedder picks the 16 passages closest to the claim; **Apertus-v1.5-8B** reads them and returns its label, cited passages and a short reason as structured JSON. Apertus is the only model that makes the decision.

On the held-out test split (976 claims, 14 vote dates never used during development), the system reaches **macro-F1 0.891** with about **3,400 prompt tokens** and a median **4.5 s** per claim on one L40S GPU. On a 60-claim test subsample, giving Apertus the whole booklet instead of retrieved passages scores **macro-F1 0.735 against 0.861** for retrieval on the same claims, while using **9× the prompt tokens** (31,400 vs 3,450) and producing 6 unparseable answers instead of 1. Retrieval makes the 8B model both more accurate and much cheaper.

2. Architecture

```
booklet PDF ─► passages ≤ 900 chars, with page + outline path      (CPU, cached)
claim ──────► top-16 passages by cross-lingual embedding           (CPU, cached per booklet)
           ─► Apertus-v1.5-8B: {reason, evidence ids, label}       (GPU, vLLM, JSON schema)
```

1. **Passages** (`booklet.py`). PyMuPDF extracts text page by page. Soft hyphens and words broken across lines are repaired. PDF text blocks are merged into passages of at most ~900 characters that never cross a page. Each passage keeps its page number and the PDF outline path of that page (vote › part › subsection), so the model and the user can see where a passage comes from. Parsed passages are cached per booklet.
2. **Retrieval** (`retrieve.py`). Claim and passages are embedded with `paraphrase-multilingual-MiniLM-L12-v2` (ONNX via fastembed, CPU). Because the embedding space is shared across languages, a French claim retrieves German passages directly, with no translation step. The top k passages are passed in document order.
3. **Verdict** (`nli.py`, `llm.py`). Apertus gets a system prompt with the three label definitions (including "a statement attributed to the wrong actor is a contradiction"), the numbered passages and the claim. Output is constrained by a JSON schema (`reason`, `evidence`, `label`); if a server does not support structured output, the client falls back to plain JSON parsing. Unparseable answers count as neutral and are reported as parse failures.
4. **Interfaces** (`cli.py`). `predict` (one claim), `batch` (JSONL), `eval` (resumable; writes predictions and metrics), `retrieval` (recall@k without an LLM). `make run` runs `eval` in Docker.

Everything that touches the GPU sits behind an OpenAI-compatible endpoint, configured by `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY`.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (official weights, bf16, no fine-tuning, no quantisation).
- **How it is used:** inference: zero-shot NLI with evidence selection.
- **Where it runs:** Modal, one NVIDIA L40S, swiss-ai's own vLLM build for Apertus 1.5 (`ghcr.io/swiss-ai/vllm_apertus_1.5_release`, pinned by digest), 128k context, prefix caching, scale-to-zero after 5 idle minutes (`modal/serve_apertus.py`). Temperature 0.
- **Evaluation runner:** `modal/run_eval.py` runs `booklet-nli eval` in a CPU container built from the project's Dockerfile, next to the GPU server, so no evaluation load runs on a laptop.

The prompt is in `src/booklet_nli/nli.py` (`SYSTEM`). It is written in English; claims and passages stay in their original languages, and the model answers its `reason` field in whichever language it chooses.

4. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (MIT licence), 1,488 claim/booklet pairs, balanced over the three labels and over all nine claim × booklet language pairs.
- **Booklets:** official explanations of the Federal Council, downloaded from bk.admin.ch on first use and cached locally; not redistributed in the repository.
- **Split.** The dataset ships without a split. We split **by vote date** (hash of the booklet publication date), so the German, French and Italian booklets of one vote are always on the same side and no booklet is seen during development and then tested on: dev = 512 pairs / 6 vote dates (all design decisions), test = 976 pairs / 14 vote dates (run once).
- No personal data is involved: claims and booklets are public political texts.

5. Evaluation

Metrics. Macro-F1 over the three labels (primary), accuracy, per-language F1. **Evidence hit**: share of non-neutral claims where at least one cited passage overlaps the annotated reference snippet (≥ 50 % of the passage's word 5-grams occur in the reference). **Context recall**: share of non-neutral claims where the reference passage was among those given to the model. Prompt and completion tokens and wall-clock seconds per claim (client side, including network).

5.1 Main result (test split, held out, run once)

Setup	n	Macro-F1	Acc	Evidence hit	Prompt tokens (mean)	s/claim (median)	Parse fail
Full booklet (baseline)	60	0.735	0.733	0.548	31,402	52.4*	6
RAG, k=16, same 60 claims	60	0.861	0.867	0.690	3,446	4.4	1
RAG, k=16 (ours)	976	**0.891**	0.891	0.716	3,424	4.5	2

* Full-booklet latency is inflated by load: 12 concurrent requests of ~31k tokens share one GPU. On dev with the same setting it was 15.5 s median, still about 3× slower than retrieval.

Confusion matrix, test, RAG k=16 (rows gold, columns predicted):

	entailment	neutral	contradiction
entailment	292	21	22
neutral	1	305	18
contradiction	13	31	273

By language (macro-F1): booklet DE 0.866, FR 0.898, IT 0.908; claim DE 0.883, FR 0.899, IT 0.890; cross-lingual pairs 0.899 (n = 641), same-language pairs 0.875 (n = 335). Cross-lingual pairs are not harder than same-language ones, which supports skipping translation.

Mean completion length is 86 tokens. The 90th-percentile latency is 6.7 s.

5.2 Design decisions (dev split)

All choices were made on dev, on the same 150-claim stratified sample.

Variant	Macro-F1	Context recall	Prompt tokens	s/claim (median)
RAG, k=8	0.850	0.82	1,818	3.0
RAG, k=16 (chosen)	**0.890**	0.92	3,354	5.0
RAG, k=16 + extra “contradiction, not neutral” rule in the prompt	0.864	–	3,404	4.7
Full booklet (n = 30)	0.582	1.00	31,785	15.5

- **Retrieval depth.** Retrieval recall@k of the reference passage on dev (338 non-neutral claims): @1 0.32, @3 0.57, @5 0.68, @8 0.77, @12 0.85, @20 0.92. Going from k=8 to k=16 fixed 9 claims and broke 3, at about 1.8× the tokens.
- **A rejected prompt change.** The most frequent dev error was contradiction predicted as neutral. One added sentence telling the model to prefer contradiction when the passages cover the point made things worse (fixed 2, broke 6; contradiction → neutral went from 7 to 9), so it was reverted. The same change flipped 11 of 150 predictions at temperature 0, which also shows how large the noise is at n = 150: differences of a few claims on dev are not meaningful.
- **Embedder.** MiniLM was chosen because it was validated end to end and runs on CPU; `multilingual-e5-large` could not be benchmarked on the 8 GB development laptop.

5.3 Error analysis (dev, k=16, 16 errors)

- 4 contradiction → neutral with the right passage in context: the model reads “the passages do not mention X” where the passage states something incompatible with X.
- 5 retrieval misses. Three are the same claim in three languages about a transition period for farmers, where retrieval returned passages about the AHV pension reform, another vote in the same multi-vote booklet. Two concern a VAT rate change (7.7 → 8.1 %) whose passage was not retrieved.
- The remaining errors are spread over entailment ↔ contradiction and neutral → contradiction.

6. Limitations

- **Retrieval is the ceiling.** About 11 % of non-neutral test claims never see their reference passage (context recall 0.89); the model cannot recover those.
- **Multi-vote booklets.** One booklet covers several votes; retrieval ignores which vote a claim is about and can mix them up.
- **Contradiction vs neutral** remains the main confusion (31 test claims), and prompt wording did not fix it.
- **Evidence hit is a proxy.** It measures n-gram overlap with one annotated snippet; a correct but different passage counts as a miss, so 0.72 is a lower bound on citation quality.
- **Latency** is measured client side over the internet against a serverless GPU, with 12 concurrent requests. A cold start (about 5 minutes) inflates the first requests of a run, which is why we report medians; the per-claim seconds in `predictions.jsonl` include it. The first 36 test claims were run from a heavily loaded laptop before the run moved to Modal; over the 940 claims run on Modal the median is 4.47 s and the mean 8.71 s.
- **Baselines.** The full-booklet comparison uses a 60-claim test subsample, not all 976 claims, to keep GPU cost low; it is directional, not precise.
- **Single run.** Each configuration was run once at temperature 0; we did not measure run-to-run variance beyond the prompt-change experiment above.

7. Reproducibility

- **Hardware:** Apertus on one NVIDIA L40S (Modal); retrieval and evaluation on a 4-vCPU Modal container; development on an 8 GB Apple M2.
- **Software:** Python 3.12, dependencies pinned in `uv.lock`; vLLM image pinned by digest.
- **Determinism:** temperature 0, sample seed 0, split by hash of vote date. Embeddings computed on different CPUs can differ in the last digits and change the order of near-tied passages (seen once in a 3-claim check: same label, 4,548 vs 4,340 prompt tokens).
- **Commands:**

```
make run                                  # Docker: balanced 60-claim test sample
make run LIMIT=976                        # full test split
modal deploy modal/serve_apertus.py       # serve Apertus
modal run modal/run_eval.py --split test --limit 976 --k 16 --name test-rag-k16-n976
```

All run directories (`predictions.jsonl` with label, cited pages, reason, tokens and seconds per claim; `metrics.json`) are in `track_2a/runs/`. A visual summary of the results is in `track_2a/docs/results.html`.

8. Next steps

- Restrict retrieval to the outline section of the vote a claim refers to, which addresses the multi-vote misses directly.
- A cross-encoder re-ranker over the top 30 passages, to raise recall at k ≤ 16 and cut tokens.
- Two-pass verification for claims predicted neutral: ask Apertus to look for an incompatible statement explicitly, only when a passage is close in embedding space.
- Run a larger Apertus model on the same pipeline to see whether the contradiction/neutral confusion is a capacity issue.
- A small web front end: paste a claim, see the verdict with highlighted booklet pages.

License

Code: Apache-2.0. This report: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Swiss AI Initiative. *Apertus-v1.5-8B.* https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- OST. *MNLI over Swiss Voting Booklets*, v1.1. https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Reimers & Gurevych (2020). *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation.* EMNLP. (`paraphrase-multilingual-MiniLM-L12-v2`)
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP. (vLLM)
- Swiss Federal Chancellery. *Erläuterungen des Bundesrates / Explications du Conseil fédéral / Spiegazioni del Consiglio federale.* https://www.bk.admin.ch